Mejoras pendientes — Guía de implementación

Agenda de contactos (Python + Tkinter + MySQL) · Documento de práctica

Este documento explica, paso a paso y con el código completo, las mejoras opcionales pendientes del proyecto. **No están aplicadas en el código fuente a propósito:** el objetivo es que tú las implementes tú mismo en tu editor para practicar.

> **Recomendación:** haz una mejora, ejecuta la app (`python python_mysql/python_mysql.py`), comprueba que todo sigue funcionando y solo entonces pasa a la siguiente. Si algo se rompe, usa `git diff` y `git checkout -- <archivo>` para volver atrás.

Orden sugerido: **5 y 7** (limpieza + .gitignore) → **1** (variables de entorno) → **2, 3, 4** (calidad de código) → **6** (confirmación al borrar).

Mejora 1: credenciales en variables de entorno (.env)

Hoy la contraseña está escrita directo en el código (connection.py, línea 34). Si subes el proyecto a GitHub, cualquier persona la ve. La solución estándar es sacar los datos de conexión del código y ponerlos en un archivo **.env** que **nunca se sube a git** (lo excluye el .gitignore de la Mejora 7).

Paso 1 — instalar la librería

```
pip install python-dotenv
```

Paso 2 — crear el archivo .env en la RAÍZ del proyecto

Junto a AGENTS.md (no dentro de python_mysql/), crea un archivo llamado exactamente **.env**:

```
DB_USER=root
DB_PASSWORD=erik123
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=clientesdb
```

Formato: `CLAVE=VALOR`, uno por línea, sin comillas y sin espacios alrededor del igual.

Paso 3 — reescribir connection.py

```
# connection.py
import os
import mysql.connector
from dotenv import load_dotenv

load_dotenv()
# load_dotenv() busca el archivo .env (desde donde ejecutes el programa)
# y carga cada CLAVE=VALOR como variable de entorno en os.environ.

class CConnection:
    def connectionBaseDeDatos():
        try:
            # os.environ.get("CLAVE", "valor_por_defecto") lee la variable;
            # si no existe, usa el valor por defecto en lugar de reventar.
            conexion = mysql.connector.connect(
                user=os.environ.get("DB_USER", "root"),
                password=os.environ.get("DB_PASSWORD", ""),
                host=os.environ.get("DB_HOST", "127.0.0.1"),
                database=os.environ.get("DB_NAME", "clientesdb"),
                port=os.environ.get("DB_PORT", "3306"),
            )
            return conexion
        except mysql.connector.Error as error:
```

```
            print("Error al conectarte a la base de datos: {}".format(error))
            return None
```

Qué aprendes aquí

- **os.environ** es un diccionario con las variables de entorno del sistema: son pares clave/valor que viven *fuera* del código.
- **load_dotenv()** es un puente: lee tu .env y lo vuelca en os.environ para que el resto del código solo use os.environ.
- Nunca subas .env a git. Si alguna vez se coló, cambia la contraseña de MySQL: ya quedó en el historial del repositorio.
- **Alternativa sin librerías:** puedes saltarte python-dotenv y definir las variables en PowerShell antes de ejecutar: `$env:DB_PASSWORD="erik123"; python python_mysql/python_mysql.py`. python-dotenv es más cómodo porque el .env viaja con el proyecto (sin subirse a git).

Mejora 2: try/finally para cerrar la conexión siempre

Hoy cada método CRUD hace: conectar → trabajar → `cone.close()`. El problema: si algo falla en medio (una consulta mal escrita, la tabla que no existe), se salta el `close()` y la conexión queda abierta de gorrón.

La sentencia **finally** se ejecuta **siempre**: tanto si todo salió bien como si hubo excepción, e incluso si la función hace return por el medio.

Patrón aplicado a mostrarClientes() (haz lo mismo con las otras 3)

```
def mostrarClientes():
    cone = None            # se declara ANTES del try: si el try no corre,
    try:                   # "cone" igual existe y finally no revienta
        cone = CConnection.connectionBaseDeDatos()
        if cone is None:
            return []
        cursor = cone.cursor()
        cursor.execute("select * from usuarios;")
        return cursor.fetchall()   # el return NO impide que finally se ejecute
    except mysql.connector.Error as error:
        print("Error al mostrar los datos {}".format(error))
        return []
    finally:
        if cone is not None:       # solo cerramos si realmente abrimos algo
            cone.close()           # esto se ejecuta SIEMPRE
```

Qué aprendes aquí

- El orden de ejecución es: try (o except) → finally, antes de devolver el valor al llamador.
- `cone = None` antes del try evita el UnboundLocalError en finally si la conexión nunca llegó a crearse.
- Alternativa avanzada (opcional): un *gestor de contexto* con `contextlib.contextmanager` que haga try/finally una sola vez y se use con `with CConnection.conexion() as cone:` en los 4 métodos. Lo dejo como reto extra.

Mejora 3: actualizarTreeview() consulta una sola vez

Hoy la función calcula `datos = CClientes.mostrarClientes()` y luego itera sobre `CClientes.mostrarClientes()` otra vez: son **dos idas y vueltas a MySQL** para lo mismo. Además, si entre una y otra consulta otra persona modifica la tabla, podrías mezclar datos de dos momentos distintos.

```
def actualizarTreeview():
```

```
    global tree
    try:
        tree.delete(*tree.get_children())   # 1) vaciar la tabla
        datos = CClientes.mostrarClientes() # 2) UNA sola consulta
        for row in datos:                   # 3) iterar sobre lo ya guardado
            tree.insert("", "end", values=row)
    except ValueError as error:
        print("Error al actualizar la tabla {}".format(error))
```

Mejora 4: quitar el commit() del SELECT y los print() de connection.py

4a) El commit() tras el SELECT

`commit()` confirma en disco los cambios de una transacción. Un SELECT no cambia nada, así que esa línea (clientes.py, línea 47) sobra: no daña, pero enseña una idea errónea. **Los commit() de INSERT, UPDATE y DELETE sí se quedan**, son los que guardan los cambios de verdad.

4b) Los print() de connection.py

`print("Conexión correcta")` se imprime en CADA operación: al abrir la app y en cada guardar/modificar/borrar/listar se llena la consola de ruido. Elimínalo. El `print` del except ya no es imprescindible porque clientes.py detecta el `None` y avisa (*"No se pudo conectar a la base de datos..."*), pero puedes conservarlo si quieres ver el detalle exacto de MySQL (contraseña mala vs. servidor caído). En un proyecto profesional ese papel lo hace el módulo `logging`, no print.

```
conexion = mysql.connector.connect(
    user=os.environ.get("DB_USER", "root"),
    password=os.environ.get("DB_PASSWORD", ""),
    host=os.environ.get("DB_HOST", "127.0.0.1"),
    database=os.environ.get("DB_NAME", "clientesdb"),
    port=os.environ.get("DB_PORT", "3306"),
)
return conexion                # sin print("Conexión correcta")
```

Mejora 5: eliminar la clase fantasma y get-pip.py

5a) La clase FormularioClientes (python_mysql.py, líneas 21-43)

Nadie crea objetos de esa clase: solo existe para declarar variables globales, y encima tiene un desajuste de nombres (`textBoxNombre` vs `texBoxNombres`), así que ni siquiera cumple su propósito. Bórrala entera y pon en su lugar declaraciones simples a nivel de módulo:

```
# Variables de la interfaz (todavía no creadas; formulario() las rellena)
win = None
groupBox = None
texBoxId = None
texBoxNombres = None
texBoxApellidos = None
combo = None
tree = None
```

Así, si alguna función se ejecuta antes de crear la ventana, recibe None y puede avisar (ya lo hacen con `if texBoxNombres is None: ...`) en lugar de crashear con NameError.

5b) El archivo get-pip.py (raíz del proyecto)

Es un instalador residual de pip que quedó en la carpeta: no forma parte de la agenda y no debe estar en el repo. Como **git lo está trackeando**, bórralo con git para que desaparezca también del repositorio:

```
git rm get-pip.py
```

Mismo caso: los `.pyc` de __pycache__ están trackeados. Desvénculalos (sin borrarlos de tu disco) con:

```
git rm -r --cached python_mysql/__pycache__
```

Después harás el commit (Mejora 7 incluye el .gitignore para que no vuelvan a entrar).

Mejora 6: confirmación antes de borrar un registro

Borrar es irreversible: con un clic accidental se pierde un cliente. `messagebox.askyesno()` muestra un cuadro con botones **Sí/No** y devuelve `True` o `False`. Solo borramos si la respuesta es True.

```
def eliminarRegistros():
    global texBoxId, texBoxNombres, texBoxApellidos
    try:
        if texBoxId is None:
            print("Los widgets no estan inicializados")
            return

        idUsuario = texBoxId.get()

        # 1) Si no hay id, no hay nada que borrar: avisar y salir.
        if idUsuario == "":
            messagebox.showwarning("Aviso",
                "Selecciona un registro de la tabla para eliminar")
            return

        # 2) Pedir confirmación. askyesno devuelve True (Sí) o False (No).
        confirmar = messagebox.askyesno("Confirmar eliminacion",
            "Seguro que quieres eliminar el registro con id {}?".format(idUsuario))

        # 3) Si el usuario dijo que no, cancelamos sin tocar la base de datos.
        if not confirmar:
            messagebox.showinfo("Informacion", "Operacion cancelada")
            return

        # 4) Solo llegamos aquí si el usuario confirmó.
        CClientes.eliminarClientes(idUsuario)
        messagebox.showinfo("Informacion", "Los datos se eliminaron correctamente")

        actualizarTreeview()
        texBoxId.delete(0, END)
        texBoxNombres.delete(0, END)
        texBoxApellidos.delete(0, END)

    except ValueError as error:
        print("Error al eliminar los datos: {}".format(error))
```

Dato clave: showinfo y showwarning solo informan (OK/Aceptar); askyesno es el que pregunta y devuelve True/False. ¿Por qué conviene verificar también que el id no esté vacío? Porque si el cuadro está vacío, `delete from usuarios where id = ''` fallaría o, peor, si algún día se construyera mal la sentencia, podría afectar a más filas de las debidas.

Mejora 7: el archivo .gitignore

Crea un archivo llamado **.gitignore** en la RAÍZ del proyecto (junto a AGENTS.md):

```
# Credenciales: variables de entorno (NUNCA se suben)
.env
.env.*
*.env

# Python
__pycache__/
*.pyc
*.pyo
.venv/
venv/
```

```
# Sistema y editores
.DS_Store
Thumbs.db
.vscode/
.idea/
```

Qué aprendes aquí

- Un patrón en .gitignore significa «esta ruta la ignora git»: el archivo existe en tu PC, pero no se commitea ni se sube.
- **Los archivos ya trackeados NO los excluye un .gitignore por sí solo.** Si .env ya estuviera en el índice, primero habría que desvincularlo: `git rm --cached .env`.
- Si el repositorio es público y la contraseña ya se subió: cámbiala en MySQL. Borrarla después no la elimina del historial de git.
- Igual que con get-pip.py, aplica a __pycache__: `git rm -r --cached python_mysql/__pycache__`.

Checklist final

- git rm get-pip.py y git rm -r --cached python_mysql/__pycache__
- Crear .gitignore en la raíz (Mejora 7)
- pip install python-dotenv y crear .env con tus credenciales (Mejora 1)
- connection.py: load_dotenv + os.environ.get; quitar print() de éxito (Mejoras 1 y 4)
- clientes.py: quitar el commit() del SELECT (Mejora 4)
- clientes.py: try/finally en los 4 métodos CRUD (Mejora 2)
- python_mysql.py: reemplazar la clase FormularioClientes por variables a nivel de módulo (Mejora 5)
- python_mysql.py: actualizarTreeview() consulta una sola vez (Mejora 3)
- python_mysql.py: askyesno en eliminarRegistros() (Mejora 6)
- Ejecutar la app y probar Guardar, Actualizar y Eliminar
- git add -A y git commit

Si algo falla tras un cambio: `git status` → `git diff <archivo>` → `git checkout -- <archivo>` para deshacer.